

WORKSHOP

The new hire.

From AI assistant to AI colleague.

A hands-on workshop for development teams.

Duration 1–2h · Monterro · June 2026

INTRO

Round the table.

Quick intro from everyone:

- 🟡 **Name & role.**
- 🔻 **What stack do you work in?**
- 🟦 **30 seconds:** something you've built with AI that you're proud of.

And — where are you today? Pick one:

The Chatter

Mostly ChatGPT or Claude in the browser.
Paste code in, paste answers back.

The Terminal Coder

One agent in the terminal — Claude Code, Cursor, Copilot agent. Plan mode, hooks, the lot.

The Speccer

Writes specs, orchestrates multiple agents in parallel. Reviews, doesn't type code.

TODAY'S THROUGH-LINE

You've just hired a new developer.

She's brilliant. But on day 1 she knows nothing about your code, your conventions, or how you work.

Today is about giving her context.

ONE WORKFLOW — ANY HARNESS

Shown in Claude. Works everywhere.

The demos today run in Claude Code — but the **workflow** is the point. Every concept is portable: AGENTS.md, skills, plan → build → verify, worktrees.

HARNESSES

Claude Code

Codex CLI

GitHub Copilot

OpenCode

Pi

Gemini CLI

Cursor

MODELS

Claude

GPT

Gemini

Open-weights (Qwen, GLM...)

Commands in this deck use Claude syntax — **your harness has an equivalent**. The workflow transfers; only the spelling changes.

WORKSHOP 1 · PART 1

01

The desk. How she thinks.

Everything she's working with sits on the desk. When it fills up — things fall off the edge.

CONTEXT WINDOW

Think of it as her desk.

What she "knows" right now lives on the desk.

- She can only reason about what's on the desk.
- When the desk fills up — things fall off the edge.
- Bigger model = bigger desk. But not infinite.
- Your job: keep the desk clean and relevant.

THE DESK · 200K TOKENS

	System message	~8K
	AGENTS.md	~2K
	Your conversation	grows
	Files she's read	~1–5K / file
	Results (tests, errors)	varies

← Full? The oldest is forgotten first.

THINK IN TOKENS

Tokenomics — every token costs.

Everything she reads, writes and remembers is tokens. Tokens are money, time — and desk space.

- 🔴 **What you send** — relevant files, not the whole repo. Every extra file costs again on every turn.
- 🔵 **What comes back** — ask for summaries and diffs, not full transcripts and dumps.
- 🟡 **What it multiplies by** — sub-agents, parallel worktrees, retries. Three colleagues with their own desks = 3× the tokens before lunch.
- 🟠 **Right model for the task** — a cheap, fast model for routine work; the strong (expensive) one for architecture and hard calls.

| Cheap per prompt ≠ cheap per workflow. **This matters more every time you multiply agents.**

THE MOST COMMON BEGINNER MISTAKE

Keep the desk clean.

- 🔴 `/context` — how full is the desk right now?
- 🔴 `/compact` — tidy up, keep a summary.
- 🔴 `/clear` — **between tasks**. The most common mistake is mixing several tasks into one session. **New task? Clear it.**
- 🔴 **Commit between phases** — the plan survives in git, not in memory.

A bigger desk (1M tokens) doesn't mean you get to skip the cleanup. And once you hit roughly **100–250K tokens** (depends on the model), she enters the **dumb zone** — quality drops, even though there's "room left." Clean before that.

/STATUSLINE

Keep an eye on it — live.

~/your-app — claude

\$ **claude**

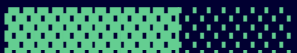
● Sonnet 4.6 · ready

// paste this prompt — she sets it up for you:

> /statusline display model, context in % and K,
cost, and branch

✓ Status line enabled

// now this shows at the bottom of your terminal:

● Sonnet 4.6 | context  62% · 124K | \$0.42 | feature/search

62% = time to start thinking about `/compact` or `/clear`.

WORKSHOP 2

02

Onboard her. AGENTS.md.

One file changes everything. It's her onboarding document.

“

You'd never put a new developer on a complex task on day 1 without context.

Why would you do it to her?

— That's the whole point.

SAME TASK, SAME AGENT

Without onboarding.

You ask her: *"Add a properties search endpoint."*

• • • ~/your-app — claude (no AGENTS.md)

Plan

- Spins up a new microservice
- Picks a different DB library
- Writes tests in the wrong runner
- Invents a folder structure

- x wrong — you live in the monolith
- x wrong — you already have an ORM
- x wrong — you use a different one
- x wrong — not your convention

She guesses. And guesses wrong.

SAME TASK · 15 LINES OF CONTEXT

With onboarding.

~/your-app — claude (with AGENTS.md)

```
$ claude "Add a search endpoint"
```

→ **Reading AGENTS.md**

→ Places the file in the **right folder** per your convention ✓

→ Uses **your ORM** and **your auth pattern** ✓

→ Writes tests in **your test runner** ✓

→ Runs tests **all green**

Same task. Same agent. **Only difference: 15 lines of context.**

ONE STANDARD, EVERY TOOL

Write it once. Every agent reads it.

TOOL	READS AGENTS.MD?	ALSO READS
Codex CLI	✓ Yes	nested AGENTS.md per package
GitHub Copilot	✓ Yes	.github/copilot-instructions.md
Cursor	✓ Yes	.cursor/rules/
Gemini CLI	✓ Yes	GEMINI.md
Windsurf	✓ Yes	.windsurf/rules/
Augment Code	✓ Yes	.augment/rules/
Claude Code	via link	CLAUDE.md — ln -s or @AGENTS.md import

The tool is replaceable. The rules travel with the code.

IMPORTANT TO UNDERSTAND

AGENTS.md is a *suggestion*,
not a **law**.

She *can* deviate. She *will* deviate sometimes.

Want to **force** something to happen — you need **hooks**.

The right prompt.

Don't use `/init` — too generic. Copy-paste **this** instead. Zero shell commands — works identically in every harness:

prompt

Analyze this codebase and create an AGENTS.md file:

1. Keep it under 150 lines (measured optimum: 100–150)
2. Cover: **WHAT** (tech stack), **WHY** (purpose), **HOW** (commands)
3. Progressive disclosure: index of `docs/` files, one-line description each
4. file:line references instead of code snippets
5. Document the 2–3 critical workflows (build, test, release) as numbered steps
6. Assume linters handle code style – no formatting rules
7. Always include these two lines:
 - Be extremely concise. Sacrifice grammar for concision.
 - At the end of each plan, list unresolved questions.

Extract recurring patterns into `docs/architectural_patterns.md`

Extract procedural know-how into `.agents/skills/<name>/SKILL.md`

GENERATE AGENTS.MD · STEP 2

The wiring.

10 seconds in the terminal. Deterministic — **nothing an agent can interpret creatively.**

macOS / Linux

```
# Claude Code reads CLAUDE.md, not AGENTS.md
$ echo "@AGENTS.md" > CLAUDE.md

# ...and .claude/skills, not .agents/skills
$ mkdir -p .claude
$ ln -s ../agents/skills .claude/skills
```

Windows (PowerShell)

```
> Set-Content CLAUDE.md "@AGENTS.md"

> New-Item -ItemType Directory -Force .claude
# copies instead of linking – re-sync after
# editing a skill
> Copy-Item -Recurse .agents\skills .claude\skills
```

This is the **harness tax** — paid until Claude Code supports the standard natively (feature request #6235, 5 200+ reactions, still open).

Checked June 2026: Claude Code still reads CLAUDE.md, not AGENTS.md. Cursor, Windsurf & Cline now read AGENTS.md natively.

PUT THESE IN EVERY AGENTS.MD

Two rules that always pay off.

"Be extremely concise."

Sacrifice grammar for concision.

→ Her plans and answers become scannable. No more walls of text.

"At the end of each plan, list unresolved questions."

→ She tells you what she *doesn't* know — instead of guessing.

The single line that saves you the most bugs.

“

Same mistake twice →
new rule in AGENTS.md.

The 2-mistake rule

Don't try to write everything up front — let mistakes teach you what to add.

TWO WAYS TO WORK WITH HER

03

Terminal or IDE?

Same agent. Two environments. Different feel.

Two ways to work with her.

Terminal Claude Code, Codex CLI, Copilot CLI, Gemini CLI

- **Full control.** Commands, hooks, skills, sub-agents.
- **Visible context.** /context, /statusline, /cost.
- **Worktrees.** Multiple agents in parallel.
- **Best for:** full features, refactoring, multi-agent.

IDE VS Code, Cursor, JetBrains

- **Visibility.** You see every file she touches, in real time.
- **Files & diff.** You see the changes immediately.
- **Lower barrier.** Good for new users.
- **Inline edits.** Select code → ask.
- **Best for:** targeted edits, review, learning.

The same AGENTS.md works in both. **Learn the terminal — that's where everything lands first.**

The workflow.

Plan → Build → Simplify → Verify. Same rhythm every time.

Plan → Build → Simplify → Verify.

- **0 · Onboard.** AGENTS.md provides context. Happens automatically.
- ▼ **1 · Plan.** Describe *what* you want (not how). She asks questions, proposes a plan.
- **2 · Build.** She implements the plan — ideally one test at a time.
- 🔲 **3 · Simplify.** "It works — make it simpler." Removes over-engineering.
- **4 · Verify.** Tests run. You review. PR.

`SHIFT + TAB × 2`

Plan mode — think before you act.

● ● ● claude · plan mode

> "Add soft delete to the Property model" *[Shift+Tab × 2]*

Planning:

1. Read the `Property model` + schema
2. Generate migration: add `deleted_at` column
3. Filter out soft-deleted rows by default
4. Add `soft_delete` + `restore` methods
5. Update existing queries
6. Write tests

Proceed?

> **yes** → implements → all tests pass ✓

Plan mode = an investigation tool. **She researches before she acts.**

Two prompts that level up plan mode.

1. "Ask me questions first"

"Add a dashboard that shows maintenance status. Poll the API every 10 seconds. **Ask me questions to clarify.**"

→ She interviews you before she plans.

2. "Give me N options"

"We need to upgrade to the next major version of our framework. Read the dependencies and config and **give me 3 options** for how to do the migration."

→ You get trade-offs on the table. Pick the one you like.

BDD · TDD · Playwright. The only way to guarantee the code does what you actually want.

2-3× QUALITY · NEARLY FREE

Give her eyes — feedback loops.

Without a feedback loop

Agent codes →
"Looks done" →
you find bugs in review

With a feedback loop

Agent codes →
runs **the tests** →
sees error → fixes → runs again → ✓

Put it in AGENTS.md: "Run tests before creating a PR."

Wire it as a hook: run the tests on session Stop — she can't skip it.

Visual: "Run Playwright and take a screenshot after every UI change." — she gets eyes.

WORKSHOP 3

Pick a real ticket.

1. Pick a ticket — bug fix or small feature.
2. Start claude in your project (with the AGENTS.md from Workshop 2).
3. Plan mode → describe the ticket → "ask me questions first".
4. Review the plan → execute.
5. Run **the tests** — all green?

Start small. **Write down what doesn't work** — that's the next line in AGENTS.md.

BONUS · DEEP DIVE

05

Train her. Skills, hooks, sub-agents, MCP, plugins.

She can work. Now we give her routines, automation, and access to your systems.

WHEN DO YOU PICK WHICH?

The decision matrix.

THE QUESTION	→ USE
Should she always know this?	AGENTS.md
Should she know this sometimes ?	Skill
Should it happen automatically ?	Hook
Does she need external systems ?	MCP
Do you need a specialist with their own desk?	Sub-agent
Want the whole package — skills + hooks + agents + MCP — installed in one go?	Plugin

REUSABLE ROUTINES

Skills — reusable slash commands.

A skill is a folder with a SKILL.md. Only the description sits on the desk — the rest loads on demand. Run it yourself with /review <file>, or let her pick it up when it matches.

skills/review/SKILL.md

```
---
name: review
description: Reviews code against our conventions
argument-hint: [file]
context: fork
agent: Explore
allowed-tools: Read, Grep, Glob
---

Review $ARGUMENTS:
1. N+1 queries?    2. Logic in the right place?
3. Auth + input validation?  4. Test coverage?

Output: ✓ good / ⚠ warning / ✗ must fix
```

name	becomes the slash command: /review
description	how she decides to load it herself
argument-hint	autocomplete hint in the / menu
disable-model-invocation	only you can run it — deploys, commits
user-invocable: false	hidden from / — background knowledge
allowed-tools	pre-approved tools while active
model / effort	cheaper model or deeper reasoning
context: fork	runs in its own context — only the summary returns
agent	who runs the fork: Explore, Plan, custom
hooks	automation scoped to the skill's run

HOOKS · REAL AUTOMATION

Hooks are *real* automation.

No LLM. No guessing.

Plain code that runs when something happens.

She writes a file → your `linter` runs.

She says "done" → **the tests run.**

.CLAUDE/SETTINGS.JSON

What a hook looks like.

```
.claude/settings.json

{
  "hooks": {
    // after a file edit:
    "PostToolUse": [{
      "matcher": "Write|Edit",
      "command": "<your-linter> --fix $FILE"
    }],
    // when she says "done":
    "Stop": [{
      "command": "<run-tests> && <your-linter>"
    }]
  }
}
```

PreToolUse	before a tool runs — can block it (guardrail)
PostToolUse	after a tool ran — lint/format every edit
UserPromptSubmit	on your prompt — inject context, validate
Stop	when she says "done" — run tests, refuse if red
SubagentStop	when a specialist finishes her task
SessionStart	when she sits down — load ticket context
PreCompact	before the desk is compacted — save state
Notification	when she needs you — ping Slack

Like a CI gate — but *before* the PR even exists. She can't skip it.

SPECIALIST COLLEAGUES · THEIR OWN DESK

Sub-agents.

A sub-agent is a specialist you bring in. She has **her own desk**, her own focus, and only returns the summary. Your main session stays clean.

 agents/reviewer.md

```
---
name: reviewer
description: Reviews code for security and conventions
tools: ["Read", "Grep", "Glob"]
model: haiku
---
```

Focus on:

1. **Security:** injection, weak input validation
2. **Performance:** unnecessary DB calls, indexes
3. **Conventions:** separation of concerns, naming
4. **Tests:** reasonable coverage, isolated units

name	her job title — how you call her in
description	when work gets handed to her — be specific
tools	her toolbox — a reviewer gets read-only
model	cheaper for routine, stronger for hard calls
skills	routines she brings to her desk
body	her job description — the system prompt

Usage: `claude --agent reviewer "Review the latest changes"` — or she delegates when the description matches.

MODEL CONTEXT PROTOCOL

MCP — connect your systems.

MCP lets her talk to your tools — not just files on disk.



GitHub

PRs, issues, reviews — straight from the terminal.



Shortcut

"Fetch SC-1234 and plan the implementation."



Context7

Up-to-date docs for frameworks, libraries, APIs.

| Add with `/mcp add github`. She reads the ticket, plans, updates status — in one flow.

HER OWN DESK

Plugins — pre-packaged skill sets.

A plugin bundles skills, hooks, sub-agents and MCP into one install — like handing her a ready-made toolkit instead of training each routine by hand.

- 🔴 **LSP** — language-server awareness: go-to-definition, real type errors, accurate refactors instead of guesswork.
- 🔴 **Review** — an automated reviewer on every change: catches regressions and style breaks before the PR.
- 🔴 **Codex** — run a second model alongside her for a different opinion on tricky tickets.

| Install one, get the whole workflow. **Start with LSP + Review** — they pay off on day one.

WHEN ONE COLLEAGUE BECOMES MANY

Three colleagues in parallel.

● ● ● three terminals — three worktrees

Terminal 1 – API

\$ **claude** -w feature-api "Build the maintenance scheduling API"

Terminal 2 – UI

\$ **claude** -w feature-ui "Build the calendar component"

Terminal 3 – Tests

\$ **claude** -w feature-tests "Integration tests for the flow"

Three colleagues working while you have a coffee. Each opens its own PR. **Remember tokenomics:** three desks = 3× the tokens.

HOW `-w` WORKS

`claude -w` — isolated by default.

- 🔴 **What it does** — `claude -w name` creates an isolated copy of the repo under `.claude/worktrees/name/` on its own branch (`worktree-name`) and starts Claude there. Your working directory is untouched.
- 🔴 **Where it branches from** — by default the remote's default branch (`origin/HEAD`, usually `origin/main`) — *not* the branch you're on. Always a clean start that matches the remote.
- 🔴 **Want a different base?** — set `"worktree": { "baseRef": "head" }` in settings to start from your local HEAD (unpushed commits come along). Running **gitflow**? Set `"baseRef": "develop"` so every worktree starts from `develop`, not `main`.
- 🔴 **Why it's useful** — run several Claude sessions in parallel without overwriting each other's files — each on its own branch, in its own directory.
- 🔴 **Cleanup** — on exit the worktree is removed automatically if nothing changed; if there are changes, Claude asks. Tip: add `.claude/worktrees/` to `.gitignore`.

“

When building is cheap —
the hard part becomes knowing **what** to build.

The new bottleneck

Ideas, not code. Understand the problem. Describe it precisely. Evaluate the result.

Common mistakes.

"I trust the code without checking"	45% of AI-generated code introduces a security flaw. ¹ Always review.
"I trust the facts without checking"	She invents statistics. Say: "double-check every claim."
"I gave her the whole repo as context"	The desk drowns. Give relevant context.
"I mix tasks in one session"	Context rot. /clear between tasks.
"It works, we ship it"	Code duplication is up 8× — refactoring has more than halved. ² Always simplify.
"We don't need AGENTS.md"	Without onboarding, she guesses — every time. Write it first.
"All agents work in my checkout"	They overwrite each other's files. One task = one worktree.

¹ Veracode GenAI Code Security Report 2025, 100+ LLMs · ² GitClear 2025, 211M changed lines 2020–2024 · Google DORA 2024: +25% AI adoption → −7.2% delivery stability.

NEXT STEPS

Homework.

- 🔴 **1. Refine your AGENTS.md.**
Add rules for the mistakes you noticed today. Commit + push.
- 🔴 **2. Work on a real ticket.**
Plan mode + worktree. Note: what worked? What didn't?
- 🔴 **3. Try `/statusline`.**
Watch how full the desk gets. Notice when it's time to `/compact`.

Thanks for joining.